Reversible Instance Normalization Experiments

Experiment Guideline

Thesis project

25 August 2026

1 Role of this document

This is the current implementation and experiment contract. The rejected historical ECML/AALTD submission is retained only in the workspace-level parent directory `../../../latex/submissions/ECML_AALTD_2026_reject`. It may provide historical context, but it is read-only, is not being resubmitted, and does not define the current protocol.

2 Forecasting setting

For user u and query date t, the model observes $X_{u,t} = X_u(t-L, t]$ and forecasts $Y_{u,t} = X_u(t, t+H]$. Chronological regions own target dates. Test1 evaluates future dates for seen users; Test2 evaluates future dates for held-out users. All compared methods share architecture, optimizer budget, batch composition, selected query dates, and original-space MSE evaluation.

3 Normalization components

The location and scale of a lookback may be chosen independently:

$$\ell_x \in \{0, \mathrm{mean}(x), x_L, \mathrm{median}(x)\}, \qquad s_x \in \{1, \mathrm{std}(x), \mathrm{MAD}(x)\}.$$

The transformed input is inverted after prediction. The core families are:

Family	Location/scale	Extra parameters
`none`	none	none
`standard`	training-global mean/std	none
`mean_only`	window mean / unit scale	none
`scale_only`	zero / window std	none
`instance`	window mean/std	none
`median_mad`	window median/MAD	none
`revin`	window mean/std	tied symmetric affine
`min`	window mean/std	untied global input/output affine

For MIN,

$$\widetilde{x} = \gamma \frac{x - \ell_x}{s_x} + \nu, \qquad \widehat{y} = s_x \left[\alpha \frac{f_\theta(\widetilde{x}) - \nu}{\gamma} + \beta \right] + \ell_x,$$

with identity initialization. MIN is global, not user-personalized.

4 Normalized backpropagation

Each applicable forward transform is paired with data-space MSE and normalized MSE:

$$\mathrm{nMSE}(\widehat{y}, y \mid x) = \frac{1}{H} \sum_{h=1}^{H} \left(\frac{\widehat{y}_h - y_h}{\mathrm{std}(x) + \varepsilon} \right)^2.$$

Only gradient weighting changes; predictions are denormalized and final tables report original-space MSE. Pairwise comparisons hold seed, model, optimizer, batching, and budget fixed.

5 Model implementation and provenance

The normalization mechanisms are implemented only in `src/proposal/normalizations.py`. Dataset windows, model composition, ordinary fitting, manifests, summaries, and plots remain respectively in `data`, `model_loading`, `training`, `pipeline`, `results`, and `visualization`.

PatchTST is source-adapted from `yuqinie98/PatchTST` revision `204c21efe0b39603ad6e2ca640ef5896646ab1a9`; DLinear is source-adapted from `cure-lab/LTSF-Linear` revision `0c113668a3b88c4c4ee586b8c5ec3e539c4de5a6`. Their isolated `external_models` packages use the common `lags, dim, horizon` boundary and are byte-identical to TimeTensors. Normalization remains outside both borrowed architectures.

6 Centering and transform ablations

Two appendix variants use non-affine instance normalization:

- `revin_last`: center on the last observation instead of the mean;
- `revin_arcsinh`: apply a reversible asinh magnitude compression after instance normalization.

Each is paired with MSE and nMSE training and compared with the matching `instance` control.

7 Seed and uncertainty contract

One seed is the single randomness source for a complete run. It fixes the user partition, data and window sampling, model initialization, and optimization. No separate partition or component seed is introduced. Reported cross-seed dispersion intentionally combines every stochastic source present in the run.

8 Profiles

Mode	Scope	Models	Budget
`test`	Electricity 504:168, narrow core smoke	PatchTST	seed 1, 2,000 steps
`small`	Traffic, Electricity, Solar; five settings	PatchTST	seeds 1–3, 10,000 steps
`full`	nine datasets; five settings	PatchTST	seeds 1–3, 10,000 steps
`ultra`	full grid	PatchTST and DLinear	seeds 1–3, 10,000 steps

The shared settings are 168:24, 336:48, and 504:168; RevIN also uses 336:96 and 336:720. Publication profiles share outputs only when configuration signatures match.

9 Execution, artifacts, and tables

Each DGX Slurm front has a matching `_selena.slurm` front that sources the same family workflow. `LOGS_ROOT` and `OUTPUTS_ROOT` default to `logs/` and `outputs/`; Selena overrides them to `logs_selena/` and `outputs_selena/`. Each front owns one family root: `core`, `nmse`, `exotic`, or `min`. The ordered relative identity below the selected output root is

```
family/dataset/L_H/backbone/normalization/loss/run_n/seed_n/
```

Normalization and loss are separate model configs. Steps, batch size, learning rate, evaluation cadence, split, and stride are pipeline configs; device and Slurm placement are runtime-only. Mode selects delayed path subsets and is not a path or computation-signature field. One seed fixes every stochastic choice.

Each `run_n/manifest.json` uses `schema_version: 1`, lists declared parameters, optional plain provenance, required artifacts, and per-seed state, and has one of the four overall statuses `not_run`, `running`, `interrupted`, or `completed`; a seed state may additionally be `ready`. Run identity contains only the schema, identity/model configs, pipeline/experiment configs, and seeds. It never fingerprints or hashes code, Slurm files, data, weights, logs, outputs, or directories; those changes are manual rerun decisions. The schema changes only for a deliberate global contract break, and `new` creates another repeat with unchanged parameters. The default `overwrite_exact` policy skips an identical completion, resumes an identical interruption, and creates the next run for a changed pipeline. The overall run remains `running` with `ready_at_utc` while finished seed states are `ready`; completion is written immediately after that configuration's producer process returns successfully. Later configuration or table failures preserve completed runs and interrupt only unfinished work. Completed manifests are authoritative, without later file hashing or synchronization checks. Tables support distinct/latest/average pipeline selection and selected/latest/distinct/average repeat selection. Explicit pipeline filters select configurations and must match even with one run; `SELECTED_RUNS.txt` stores only automatic or pinned exact repeats. Reports live at `reports/family/mode` below the selected output root and record requested filters and obtained input manifests.

Slurm workflows never submit a publisher or execute Git commands. After any terminal state, including failure, cancellation, or timeout, the user runs `bash publish_job.sh <job-id>` from the project root. The script verifies `main`, sources `$HOME/codes/proxy.sh` (or `PROXY_SCRIPT_PATH`), and fast-forward pulls `origin/main` before artifact selection or staging. Lightweight publication selects the exact local or Selena stdout/stderr pair when a job ID is supplied, all log trees otherwise, and only aggregate `outputs*/reports/` artifacts. Detailed publication adds all non-binary outputs and diagnostics. Both tiers exclude `*.pt`, `*.npy`, and `*.cbm`. The script then pushes `origin/main` without opening a pull request. A non-fast-forward pull stops without a merge commit, and unrelated staged paths remain outside the publisher commit.

Ninety-four reconstructable configurations were migrated from the earlier flat-backbone implementation. Their source trees remain under `outputs/archive/legacy_pre_schema_v1_2026-08-07`, but the migrated `outputs/core` manifests also predate the pinned source packages above. They are retained only as analyzed historical evidence and are not eligible for current model comparisons. Reproduction must allocate new runs for every affected PatchTST or DLinear configuration before current tables are used.

Tables report mean and seed standard deviation, explicit row multipliers, validation-selected policies, and a clearly labeled optimistic test oracle. The oracle is diagnostic only. Dataset configuration uses shared top-level fields and project-scoped overrides. For `drop_users`, null inherits, an empty list retains every CSV user, and a nonempty run value replaces the configured default.

10 Practical validation

Run external-package forwards, result-format checks, and Slurm workflow tests locally. Training is a remote cluster task. The first current-package submission must use a new-run policy so the superseded completed manifests are not selected as reusable computations.